

华中科技大学

2026

逻辑与计算机系统设计 · 实验报告 ·

专 业：智能科学与技术

班 级：智科 2402

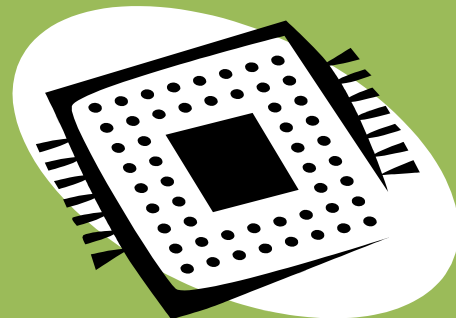
学 号：U202414845

姓 名：李金轩

电 话：13781772037

邮 件：U202414845@hust.edu.cn

完成日期：2026-06-11



计算机科学与技术学院

1 运动码表系统设计

1.1 设计要求

利用 logisim 平台中现有运算部件构建一个运动码表系统设计，主要包括 7 段数码管驱动电路、2 选 1 选择器（1 位或 16 位）、无符号比较器设计（4 位或 16 位）、并行加载寄存器（4 位或 16 位）、BCD 计数器状态机及输出函数、BCD 计数器（1 位十进制）、码表计数器（4 位十进制）、码表显示驱动、码表控制器状态机及输出函数、码表控制器、运动码表数据通路的设计。最终系统应具备计时、存储时间、复位和显示时间的功能。

1.2 方案设计

1.2.1 7 段数码管驱动电路设计

首先观察数码管，分析得知如果要对 0-9 数字的正常显示，至少需要 10 个状态表示，因此输入引脚数至少需要 4 位。由图可见，数字 1-7 是需要控制显示的部分，因此需要 7 位输出引脚来表示不同状态。数字 8 表示的小数点不用表示。

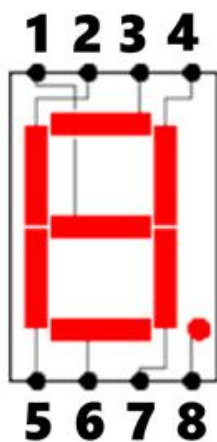


图 1-1：7 段数码管结构示意图

华中科技大学课程实验报告

电路引脚及功能如表 1-1 所示：

表 1-1 片引脚与功能描述

引脚	输入/输出	位宽	功能描述
X3~X0	输入	1	4 位 BCD 码输入
Seg1~Seg7	输出	1	7 位数码管驱动信号

因此，在设计电路时候，首先确定十个状态对应的 BCD 码是什么，根据 BCD 码（8421 码）的规则得出十个状态是 0000 - 1001，对应数字 0-9，根据不同的 BCD 码调节 7 个输出引脚的状态，1 表示显示，0 表示隐藏，就能得到正确结果。因此只需要通过工程页面的分析组合逻辑电路功能，调整 10 个状态对应的真值表即可，剩余的 6 个状态随意即可。如图 1-2 所示：

X3	X2	X1	X0	Seg_1	Seg_2	Seg_3	Seg_4	Seg_5	Seg_6	Seg_7
0	0	0	0	0	1	1	1	1	1	1
0	0	0	1	0	0	0	1	0	0	1
0	0	1	0	1	0	1	1	1	1	0
0	0	1	1	1	0	1	1	0	1	1
0	1	0	0	1	1	0	1	0	0	1
0	1	0	1	1	1	1	1	0	0	1
0	1	1	0	1	1	1	1	0	1	1
0	1	1	1	0	0	1	1	0	0	1
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0	0	1	1	1	1	1	1
1	0	1	1	0	0	0	1	0	0	1
1	1	0	0	0	1	0	1	1	1	0
1	1	0	1	1	0	1	1	0	1	1
1	1	1	0	1	1	1	0	1	0	1
1	1	1	1	1	1	1	0	0	1	1

a

图 1-2：7 段数段管电路真值表

随后点击生成电路自动生成电路就能生成预期电路。但是这样生成的电路会出现“0”型险象，当 $X0=0$, $X1=1$, $X3=0$ 时，Seg1 表达式会变为 $\sim X2+X2$ ，如图 1-3 所示，因此增加冗余项后消除险象，如图 1-4 所示。

		X1, X0			
		00	01	11	10
X3, X2	00	0	0	1	1
	01	1	1	0	1
	11	1	1	1	1
	10	1	1	0	0

$X3 X2 X1 + X2 X1 + X2 X0 + X3 X1 + X3 X2$

图 1-3 Seg1 输出

		X1, X0			
		00	01	11	10
X3, X2	00	0	0	1	1
	01	1	1	0	1
	11	1	1	1	1
	10	1	1	1	1

$X2 X1 + X2 X1 + X2 X0 + X3$

图 1-4 无险象的 Seg1 输出

华中科技大学课程实验报告

1.2.2 2选1选择器设计（1位）

本关要利用基本逻辑门构建1位的2路选择器，电路的引脚及功能如表1-2所示。

表 1-2 2选1选择器（1位）电路引脚与功能描述

信号	I/O	位宽	说明
X0	输入	1	2路输入的第0路
X1	输入	1	2路输入的第1路
Sel	输入	1	选择控制端
Out	输出	1	$Out = (Sel == 0) ? X0 : X1;$

根据输出表达式 $OUT = Sel \cdot X1 + X0 \cdot \sim Sel$ ，画出电路图：

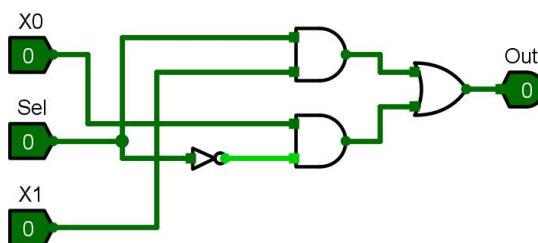


图 1-5：2选1选择器（1位）电路图

但分析表达式得知，当 $X1=X0=1$ 时候出现“0”型险象，因此增加冗余项 $X0X1$ ，修正过后的电路图 1-6：

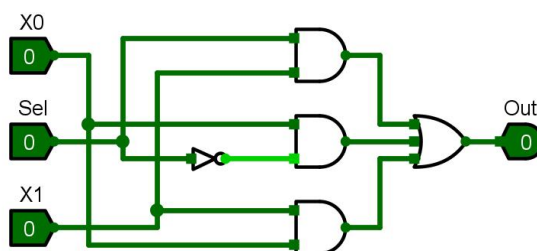


图 1-6：消除险象后的二路选择器电路图（1位）

1.2.3 2选1选择器设计（16位）

根据2选1选择器设计（1位）很容易想到，16位的设计无非就是将输入的16位拆成16个1位，用16个1位选择器即可实现，这里需要用到分线器（Splitter）分离

华中科技大学课程实验报告

开 X0 和 X1 的 16 位数据，并分别置入 16 个 1 位 2 路选择器的输入端，结果输出到 out 端即可。如图 1-7 所示：

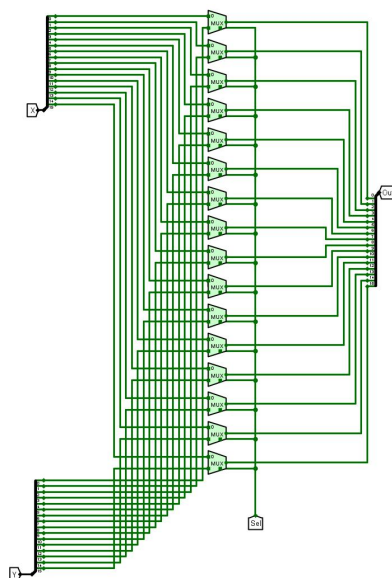


图 1-7 2 选 1 选择器（16 位）电路图

1.2.4 无符号比较器设计（4 位）

本关要设计实现四位无符号数的比较器。电路引脚及功能如表 1-3 所示：

表 1-3 无符号比较器（4 位）电路引脚与功能描述

信号	I/O	位宽	说明
X3~X0	输入	1	4 位输入 X
Y3~Y0	输入	1	4 位输入 Y
Great	输出	1	X 大于 Y
Equal	输出	1	X 等于 Y
Less	输出	1	X 小于 Y

四位无符号比较器可以分解为四个 1 位从高到底比较，当 $X > Y$ ，若满足 $X_3 > Y_3$ ，或 $X_3 = Y_3$ 且 $X_2 > Y_2$ ，或 $X_3 = Y_3$ 且 $X_2 = Y_2$ 且 $X_1 > Y_1$ ，或 $X_3 = Y_3$ 且 $X_2 = Y_2$ 且 $X_1 = Y_1$ 且 $X_0 > Y_0$ 。 $X_3 < Y_3$ 翻转 X,Y 即可。当且仅当 $X_3 = Y_3$ 且 $X_2 = Y_2$ 且 $X_1 = Y_1$ 且 $X_0 = Y_0$ 时， $X = Y$ 成立。根据这些关系可以写出 Great、Equal、Less 的逻辑表达式（与运算用字母间空格表示，非运算用 ‘~’ 表示，或运算用 ‘+’ 表示）：

华中科技大学课程实验报告

$$\text{Great} = X_0 \sim Y_3 \sim Y_2 \sim Y_1 \sim Y_0 + X_1 \sim Y_3 \sim Y_2 \sim Y_1 + X_1 X_0 \sim Y_3 \sim Y_2 \sim Y_0 + X_2 \sim Y_3 \sim Y_2 + X_2 X_0 \sim Y_3 \sim Y_1 \sim Y_0 + X_2 X_1 \sim Y_3 \sim Y_1 + X_2 X_1 X_0 \sim Y_3 \sim Y_0 + X_3 \sim Y_3 + X_3 X_0 \sim Y_2 \sim Y_1 \sim Y_0 + X_3 X_1 \sim Y_2 \sim Y_1 + X_3 X_1 X_0 \sim Y_2 \sim Y_0 + X_3 X_2 \sim Y_2 + X_3 X_2 X_0 \sim Y_1 \sim Y_0 + X_3 X_2 X_1 \sim Y_1 + X_3 X_2 X_1 X_0 \sim Y_0。$$

$$\text{Equal} = \sim X_3 \sim X_2 \sim X_1 \sim X_0 \sim Y_3 \sim Y_2 \sim Y_1 \sim Y_0 + \sim X_3 \sim X_2 \sim X_1 X_0 \sim Y_3 \sim Y_2 \sim Y_1 Y_0 + \sim X_3 \sim X_2 X_1 \sim X_0 \sim Y_3 \sim Y_2 Y_1 \sim Y_0 + \sim X_3 \sim X_2 X_1 X_0 \sim Y_3 \sim Y_2 Y_1 Y_0 + \sim X_3 X_2 \sim X_1 \sim X_0 \sim Y_3 Y_2 \sim Y_1 \sim Y_0 + \sim X_3 X_2 \sim X_1 X_0 \sim Y_3 Y_2 \sim Y_1 Y_0 + \sim X_3 X_2 X_1 \sim X_0 \sim Y_3 Y_2 Y_1 \sim Y_0 + \sim X_3 X_2 X_1 X_0 \sim Y_3 Y_2 Y_1 Y_0 + X_3 \sim X_2 \sim X_1 \sim X_0 Y_3 \sim Y_2 \sim Y_1 \sim Y_0 + X_3 \sim X_2 \sim X_1 X_0 Y_3 \sim Y_2 \sim Y_1 Y_0 + X_3 \sim X_2 X_1 \sim X_0 Y_3 \sim Y_2 Y_1 \sim Y_0 + X_3 \sim X_2 X_1 X_0 Y_3 \sim Y_2 Y_1 Y_0 + X_3 X_2 \sim X_1 \sim X_0 Y_3 Y_2 \sim Y_1 \sim Y_0 + X_3 X_2 \sim X_1 X_0 Y_3 Y_2 \sim Y_1 Y_0 + X_3 X_2 X_1 \sim X_0 Y_3 Y_2 Y_1 \sim Y_0 + X_3 X_2 X_1 X_0 Y_3 Y_2 Y_1 Y_0。$$

$$\text{Less} = \sim X_3 \sim X_2 \sim X_1 \sim X_0 Y_0 + \sim X_3 \sim X_2 \sim X_1 Y_1 + \sim X_3 \sim X_2 \sim X_0 Y_1 Y_0 + \sim X_3 \sim X_2 Y_2 + \sim X_3 \sim X_1 \sim X_0 Y_2 Y_0 + \sim X_3 \sim X_1 Y_2 Y_1 + \sim X_3 \sim X_0 Y_2 Y_1 Y_0 + \sim X_3 Y_3 + \sim X_2 \sim X_1 \sim X_0 Y_3 Y_0 + \sim X_2 \sim X_1 Y_3 Y_1 + \sim X_2 \sim X_0 Y_3 Y_1 Y_0 + \sim X_2 Y_3 Y_2 + \sim X_1 \sim X_0 Y_3 Y_2 Y_0 + \sim X_1 Y_3 Y_2 Y_1 + \sim X_0 Y_3 Y_2 Y_1 Y_0。$$

输入后自动生成电路即可，如图 1-8 所示：

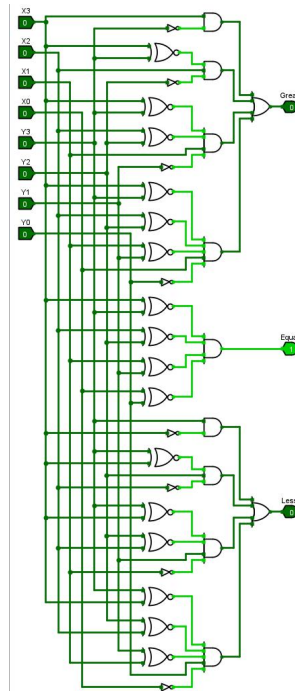


图 1-8 无符号比较器（4 位）电路图

1.2.5 无符号比较器设计（16 位）

本关的设计可以将 4 位看为一个整体，即 X0-X3 看为 X0，X4-X7 看为 X1，X8-X11 看为 X2，X12-X15 看为 X3。同理可得到 Y0-Y3，如图 1-9 所示：

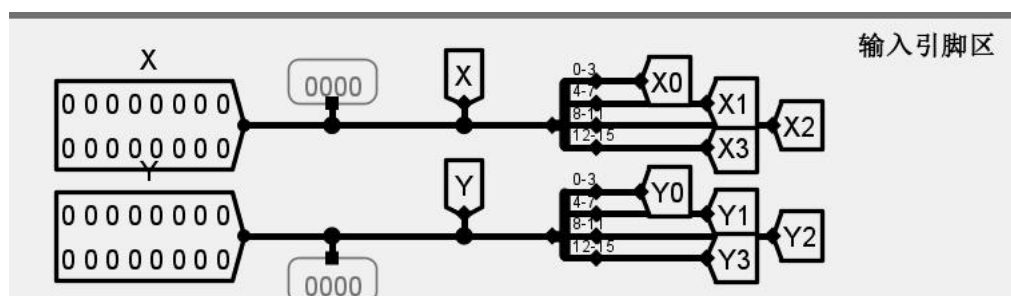


图 1-9 输入引脚设置

因此接下来的分析可以和无符号比较器（4 位）对照，只不过位之间的比较通过无符号比较器（4 位）完成，利用每个比较器输出的 Great/Equal/Less 信号可以完成无符号比较器（16 位）设计，如图 1-10 所示：

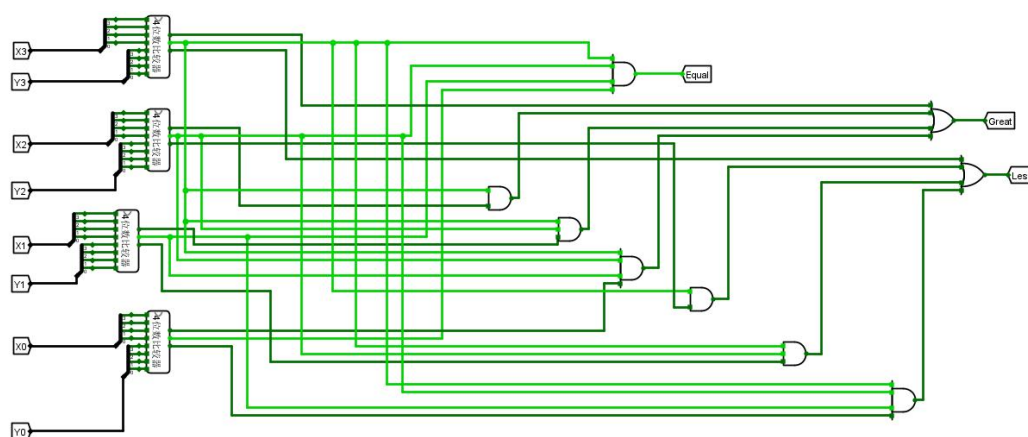


图 1-10 无符号比较器（16 位）电路图

1.2.6 并行加载寄存器设计（4 位）

本关需要构建由 4 个 D 触发器构成的 4 位基本寄存器，要求上升沿触发，具有高电平使能端。寄存器是由具有存储功能的触发器组合而成的，其中一个触发器存储一位二进制代码。因此存放 4 位二进制代码的寄存器，需要 4 个 D 触发器构成。上升沿触发和高电平使能端的要求则规定了 D 触发器的输入端设置。电路引脚及功能如

华中科技大学课程实验报告

表 1-4 所示:

表 1-4 并行加载寄存器（4 位）电路引脚与功能描述

信号	I/O	位宽	说明
Clock	输入	1	时钟脉冲，上升沿有效
Din	输入	4	寄存数据输入端
En	输入	1	使能输入端，高电平有限
Q	输出	4	寄存数据输出端：En=1、Clock ↑ 时 Q=Din

根据 D 触发器的状态变化表达式： $Q_{n+1} = D$ ，只需要把输入用分线器分开，分别输入 4 个 D 触发器的“D”输入端，并将这 4 个 D 触发器的次态输出按位输出到 Q 即可。电路图如图 1-11 所示：

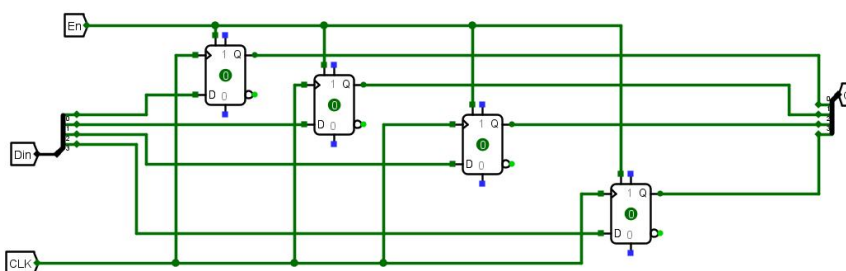


图 1-11 并行加载寄存器（4 位）电路图

1.2.7 并行加载寄存器设计（16 位）

本关设计思想和无符号比较器（16 位）有异曲同工地方，都可以看作四个部分组装，本关可以将输入用分线器分为 4 部分，分别输入到 4 个并行加载器（4 位）中，输出结果再用分线器合成为 16 位即可。电路如图 1-12 所示：

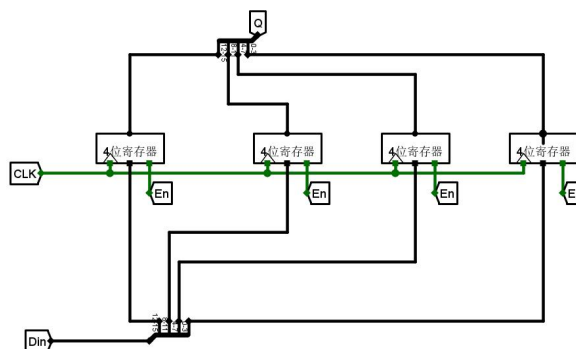


图 1-12 并行加载寄存器（16 位）电路图

华中科技大学课程实验报告

1.2.8 BCD 计数器状态机设计

本关需要设计 BCD 状态机，电路引脚及功能如表 1-5 所示：

表 1-5 BCD 计数器状态机设计电路引脚与功能描述

信号	I/O	位宽	说明
S3-S0	输入	4	当前状态 S
N3-N0	输入	4	次态输出 N, $N=S+1$

本关要求完成状态转换真值表，由 $N = S+1$ 得出真值表，假定使用的是 D 触发器，则 $ln1 = N3, ln2 = N2, ln3 = N3, ln4 = N4, ln5$ 储存进位信息，由此得到的真值表如图 1-13 所示：

当前状态(现态)					输入信号								下一状态 (次态)				
S3	S2	S1	S0	现态 10进制	In1	In2	In3	In4	In5	In6	In7	In8	次态 10进制	N3	N2	N1	N0
0	0	0	0	0	0	0	0	1	0				1	0	0	0	1
0	0	0	1	1	0	0	1	0	0				2	0	0	1	0
0	0	1	0	2	0	0	1	1	0				3	0	0	1	1
0	0	1	1	3	0	1	0	0	0				4	0	1	0	0
0	1	0	0	4	0	1	0	1	0				5	0	1	0	1
0	1	0	1	5	0	1	1	0	0				6	0	1	1	0
0	1	1	0	6	0	1	1	1	0				7	0	1	1	1
0	1	1	1	7	1	0	0	0	0				8	1	0	0	0
1	0	0	0	8	1	0	0	1	0				9	1	0	0	1
1	0	0	1	9	0	0	0	0	1				0	0	0	0	0

图 1-13 BCD 计数器状态转换真值表

填写完成后，在 Excel 表格的“状态转换函数自动生成”下自动生成次态输出逻辑表达式。其中， $N3 = \sim S3 S2 S1 S0 + S3 \sim S2 \sim S1 \sim S0$ ， $N2 = \sim S3 \sim S2 S1 S0 + \sim S3 S2 \sim S1 + \sim S3 S2 \sim S0$ ， $N1 = \sim S3 \sim S1 S0 + \sim S3 S1 \sim S0$ ， $N0 = \sim S3 \sim S0 + \sim S2 \sim S1 \sim S0$ 。然后在 Logisim 中自动生成电路即可，电路图如图 1-14 所示：

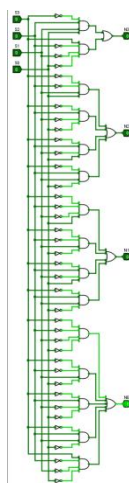


图 1-14 BCD 计数器状态机电路图

华中科技大学课程实验报告

1.2.9 BCD 计数器输出函数设计

本关要求实现 BCD 计数器输出函数设计，电路引脚及功能如表 1-6 所示：

表 1-6 BCD 计数器输出函数设计电路引脚与功能描述

信号	I/O	位宽	说明
S3-S0	输入	4	当前状态 S
Cout	输入	1	进位输出，S=9 时，Cout=1

当且仅当输入为 1001 时候，输出结果为 1，因此 $Cout = S3 \sim S2 \sim S1 S0$ ，据此电路如图 1-15 所示：

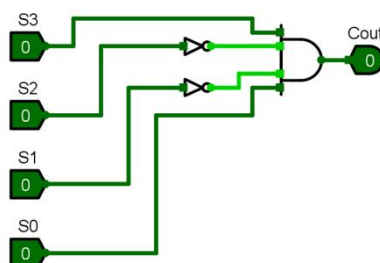


图 1-15 BCD 计数器输出函数电路图

1.2.10 BCD 计数器设计（1 位十进制）

下面要利用已经设计好的 BCD 计数器的状态机、输出函数，采用 D 触发器构建最终的 BCD 计数器。电路引脚及功能如表 1-7 所示：

表 1-7 BCD 计数器（1 位十进制）电路引脚与功能描述

信号	I/O	位宽	说明
CLK	输入	1	时钟输入
Rst	输入	1	异步复位信号，为 1 时 $Q=0$
En	输入	1	使能信号，为 1 时进行计数， $Q=Q+1$
Q	输出	4	计数器计数输出
Count	输出	1	进位输出，计数到 9 时输出为 1

核心思想是利用 D 触发器实现状态覆盖，也就是 $Q=Q+1$ 。每次 CLK 从 0 变为 1 表示将 $Q+1$ 赋给 Q，CLK 从 1 变为 0 表示将 $Q+1$ 存到 D 触发器中，当 $Q+1 = 1001$ 时，

CLK 从 0 变 1 同时输出进位 1。根据之前实现的状态转换和输出函数的输入和输出，电路如图 1-16 所示：

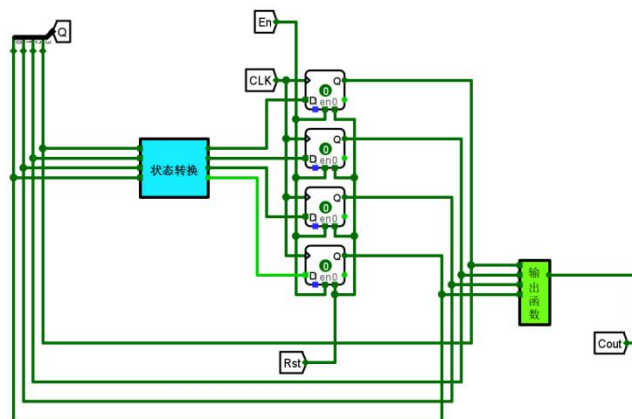


图 1-16 BCD 计数器（1 位十进制）

1.2.11 码表计数器设计（4 位十进制）

本关要求实现 4 个十进制的计数器，输出一个 16 位的结果。因此需要考虑如何实现满 10 进 1 这个目的。在计数器（1 位十进制）中 CLK 从 0 变 1 时 Q 会变化，因此需要做到低位是 9 时，高位的 CLK 是 0；低位是 0（10）时，高位的 CLK 是 1，且一直到下一个低位为 9 前都会继续保持 1。低位为 9，cout 是 1，其他时候 cout 都是 0。因此可以将低位的 cout 取非连接到高位 CLK，最终电路如图 1-17 所示：

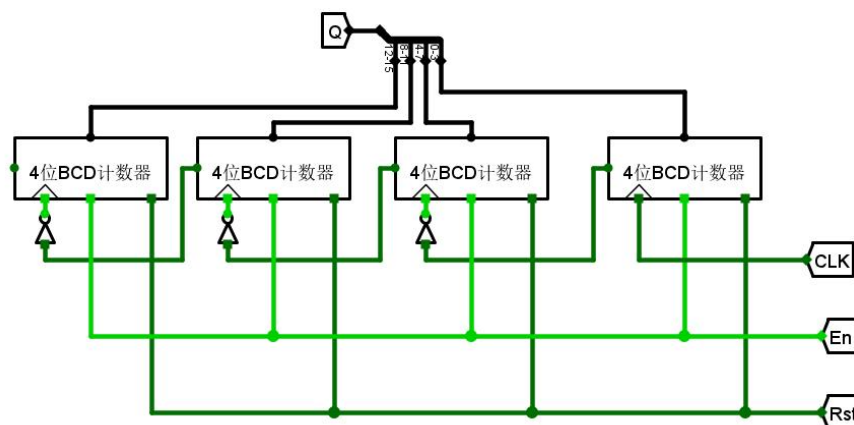


图 1-17 码表计数器电路图

1.2.12 码表显示驱动设计

本关要利用第 1 关的 7 段数码管驱动电路级联构建 4 位十进制显示驱动电路。电路引脚及功能如表 1-8 所示：

表 1-8 码表显示驱动电路引脚与功能描述

信号	I/O	位宽	说明
Din	输入	16	4 位十进制（16 位 BCD）
DisplInfo	输出	32	4 个 8 段（7 段+小数点）数码驱动信号

若只将四个 7 段数码管驱动输出连接，会少 4 位输出，由于实现的数码管驱动只会输出 7 位控制信号而不会输出小数点信号，因此需要额外添加小数点信号。根据测试电路中显示的，小数点在第二位数字后面，因此只需要将第二位的小数点输出信号置为 1 即可，其他位置的小数点输出信号置为 0。电路如图 1-18 所示：

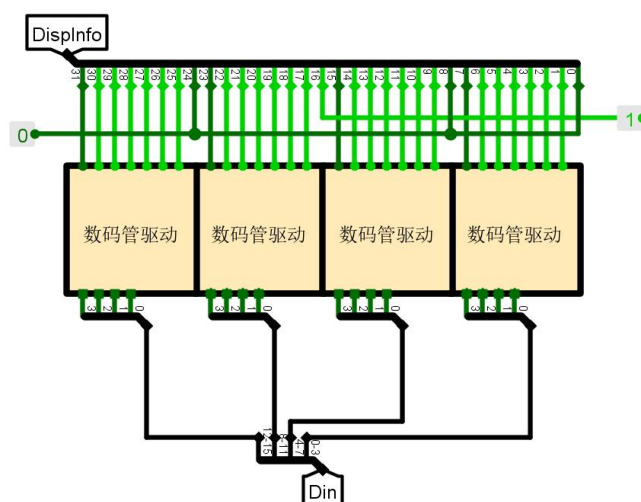


图 1-18 码表显示驱动电路

1.2.13 码表控制器状态机设计

本关要设计码表控制器的状态机（即状态转换组合逻辑）。电路引脚及功能如

华中科技大学课程实验报告

表 1-9 所示：

表 1-9 码表控制器状态机电路引脚与功能描述

信号	I/O	位宽	说明
start	输入	1	开始计时
stop	输入	1	停止计时
store	输入	1	存储计时记录
resert	输入	1	计时复位，记录恢复为 99.99
newrecord	输入	1	新最好成绩记录
S2-S0	输入	3	现态 S（present state）
N2-N0	输出	3	次态 N（next state）

题目要求根据状态图完成状态转换真值表，状态图如图 1-19 所示：

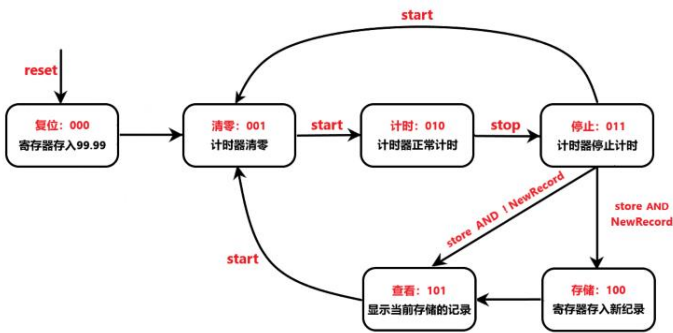


图 1-19 码表控制器状态机状态转移图

根据这一状态转移图，可以在“同步时序电路状态转换表.xlsx”中完成状态转换真值表，如图 1-20 所示：

当前状态(现态)					输入信号						下一状态 (次态)				
S3	S2	S1	S0	现态 10进制	start	stop	store	resert	NewRecord		次态 10进制	N3	N2	N1	N0
0	0	0	0	0	0	0	0	0	0		1	0	0	0	1
0	0	0	1	1	1	0	0	0	0		2	0	0	1	0
0	0	0	1	1	0	0	0	0	0		1	0	0	0	1
0	0	1	0	2	0	1	0	0	0		3	0	0	1	1
0	0	1	1	3	1	0	0	0	0		1	0	0	0	1
0	0	1	1	3	0	0	1	0	1		4	0	1	0	0
0	0	1	1	3	0	0	1	0	0		5	0	1	0	1
0	1	0	0	4	0	0	0	0	0		5	0	1	0	1
0	1	0	1	5	1	0	0	0	0		1	0	0	0	1
0	1	0	1	5	0	0	0	0	0		5	0	1	0	1
0	0	1	0	2	0	0	0	0	0		2	0	0	1	0
0	0	1	1	3	0	0	0	0	0		3	0	0	1	1
								1			0	0	0	0	0

图 1-20 码表控制器状态转换真值表

华中科技大学课程实验报告

需要注意到的隐含状态转换条件：（1）在状态为 1（清零）时，若不给出 start 指令，则会一直维持状态 1；（2）状态为 4（存储）时，若不给出任何指令，则会直接跳转到状态 5（查看）；（3）只有给出 reset 指令时，才会转移到状态 0（复位），其它任何条件下都不会转移到状态 0。

之后与 BCD 计数器状态机一样，利用 Excel 表格自动生成次态输出逻辑表达式。其中 N2, N1, N0 满足以下逻辑表达式（与运算用字母间空格表示，非运算用 ‘~’ 表示，或运算用 ‘+’ 表示）：

$$N2 = \sim \text{start} \sim \text{reset} S2 \sim S1 + \sim \text{reset} S2 \sim S1 \sim S0 + \sim \text{start} \text{store} \sim \text{reset} \sim S2 S1 S0$$

$$N1 = \sim \text{start} \sim \text{store} \sim \text{reset} \sim S2 S1 + \sim \text{reset} \sim S2 S1 \sim S0 + \text{start} \sim \text{reset} \sim S2 \sim S1 S0$$

$$N0 = \sim \text{start} \sim \text{reset} \sim S1 + \sim \text{reset} \sim S1 \sim S0 + \sim \text{start} \sim \text{store} \sim \text{reset} \sim S2 S0 + \sim \text{start} \sim \text{reset} \sim \text{NewRecord} \sim S2 S0 + \sim \text{reset} S2 \sim S1 + \text{stop} \sim \text{reset} \sim S2 \sim S0 + \text{start} \sim \text{reset} \sim S2 S1 S0。$$

电路如图 1-21 所示：

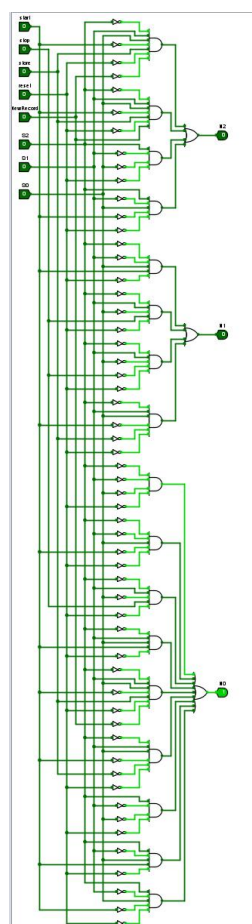


图 1-21 码表控制器状态机电路图

1.2.14 码表控制器输出函数设计

本关要设计码表控制器的输出函数组合逻辑（Moore 型输出），需要结合运动码表系统数据通路完成码表控制器输出函数真值表。电路引脚及功能如表 1-10 所示。

表 1-10 码表控制器输出函数电路引脚与功能描述

信号	I/O	位宽	说明
S2-S0	输入	3	当前状态 S（PS）
SDsel	输出	1	好成绩记录的选择
SDen	输出	1	保存成绩记录的寄存器使能
DPsel	输出	1	显示成绩记录的选择
TMen	输出	1	码表计时器使能
TMreset	输出	1	码表计时器复位

根据测试样例得出输出函数真值表，如图 1-22 所示：

当前状态(现态)					输入信号												
S3	S2	S1	S0	现态 10进制	start	stop	store	reset	#REF!	#REF!	0	0	SDsel	SDen	DPsel	TMen	TMreset
0	0	0	0	0										1	1		1
0	0	0	1	1										1	1		1
0	0	1	0	2											1	1	
0	0	1	1	3											1		
0	1	0	0	4									1	1	1		
0	1	0	1	5													

图 1-22 码表控制器输出函数真值表

用 Excel 表格的“输出函数自动生成”自动生成输出函数如下：
 $SDsel = S2 \sim S1 \sim S0$, $SDen = \sim S1 \sim S0$, $DPsel = \sim S2 + \sim S1 \sim S0$, $TMen = \sim S2 S1 \sim S0$, $TMreset = \sim S2 \sim S1$ 。电路如图 1-23 所示：

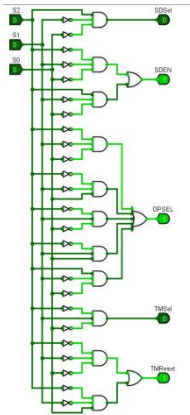


图 1-23 码表控制器输出函数电路图

1.2.15 码表控制器设计

本关内容：根据同步时序逻辑电路基本模型，利用已经设计完成的码表控制器的状态机（状态转换组合逻辑）、输出函数，采用 D 触发器（或者寄存器）构建最终的码表控制器，完成有限状态机的组装。电路引脚及功能如表 1-11 所示：

表 1-11 码表控制器电路引脚与功能描述

信号	I/O	位宽	说明
CLK	输入	1	时钟脉冲
start	输入	1	开始计时
stop	输入	1	停止计时
store	输入	1	存储计时记录
reset	输入	1	计时复位，记录恢复为 99.99
newrecord	输入	1	新的成绩记录
SDsel	输出	1	成绩记录的选择
SDen	输出	1	保存成绩记录的寄存器使能
DPsel	输出	1	显示成绩记录的选择
TMen	输出	1	码表计数器使能
TMreset	输出	1	码表计数器复位

由于本关是两个已经实现的电路封装的集合，状态机的输入是 start,stop,store,reset,NewRecord,S0-S2，输出是 N0-N2，输出函数的输入是 S0-S2，输出是 SDsel,SDen,DPsel,TMen,TMreset。因此需要做的就是利用 D 触发器存储中间过程的 N0-N2，一边传回状态机作为现态，一边传到输出函数记录。电路如图：

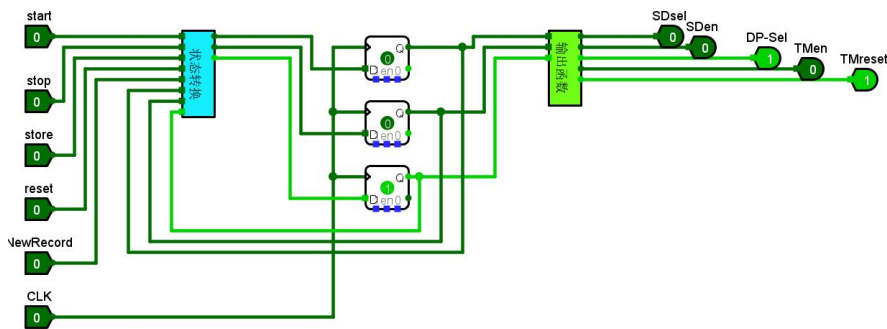


图 1-24 码表控制器电路图

1.2.16 运动码表数据通路设计（系统集成）

本关内容：利用前面关卡中设计完成的相关部件（选择器、比较器、寄存器、计数器、码表控制器等）构建运动码表系统数据通路。电路引脚及功能如表 1-12 所示：

表 1-12 运动码表数据通路电路引脚与功能描述

信号	I/O	位宽	说明
CLK	输入	1	时钟脉冲
TestMode	输入	1	在线评测开关，0 为本地模式，1 为在线模式
start	输入	1	开始计时信号
stop	输入	1	停止计时信号
store	输入	1	存储计时记录信号
reset	输入	1	计时复位信号，记录恢复为 99.99
Time	输出	16	计时成绩或者成绩记录

模块用处：用码表控制器将输入的信号转化为其他模块需要用到的信号，用码表计数器记录时间，用 16 位并行加载寄存器存储成绩，用码表显示器显示时间。

本关将输入的信号转化为六个信号：（1）SD-SEL：16 位寄存器 SD 前的 MUX，0：将当前计时值存入 SD（2）SD-EN：16 位寄存器 SD，1：允许写入 SD（锁存当前成绩或最好成绩）（3）DP-SEL：码表显示的 MUX，0：数码管显示当前计时值，1：数码管显示存储的成绩。（4）TM-EN：计时器 TM，1：计时器开始/继续计数，0：计时器暂停（5）TM-Reset:计时器 TM，1：计时器清零复位。根据信号连线，由此电路如图 1-25：

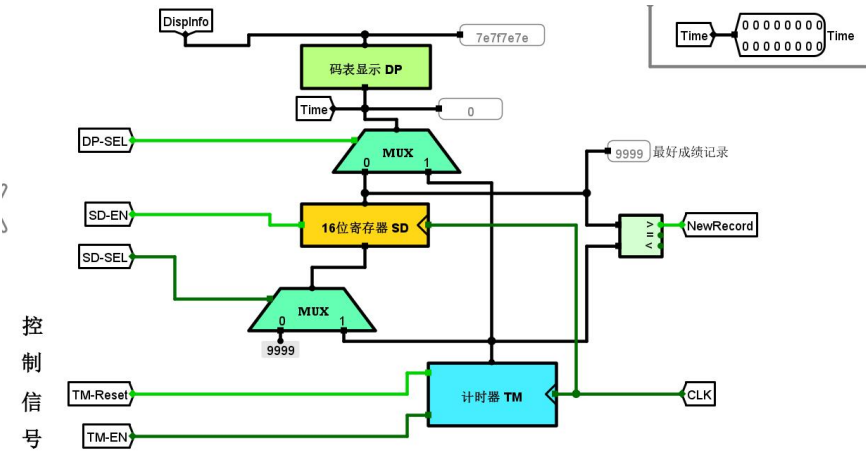


图 1-25 运动码表电路图

1.3 实验步骤

(1) 步骤一：根据 1.2 方案设计，设计每个关卡的电路图。

(2) 步骤二：依次测试是否能通过测试电路，包括数码管驱动测试，2 路选择器自动测试，16 位无符号比较器自动测试，码表计数器自动测试，运动码表测试，接下来展示码表计数器测试和运动码表测试：

①在码表计数器测试中键入 Ctrl+K 会自动测试：

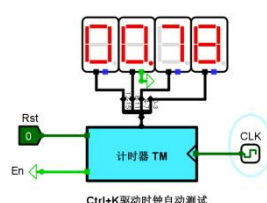


图 1-26 码表计数器测试

②运动码表的本地测试所测试的功能有计数，复位，存储。当 start 置 1，点按 clk 可模仿计时；resert 置 1 之后就会复位；stop 置 1 就会停止；store 置 1 就会存储：

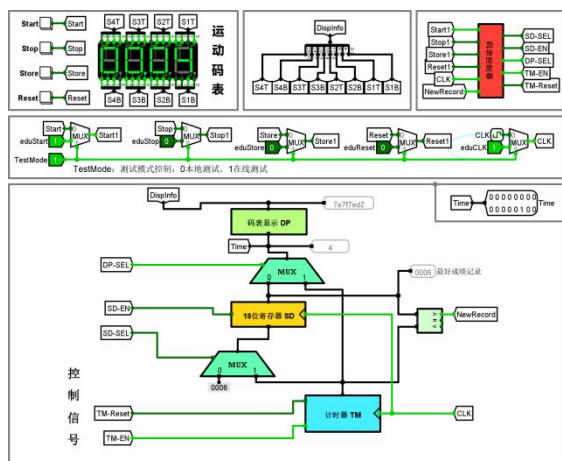


图 1-27 运动码表计时功能

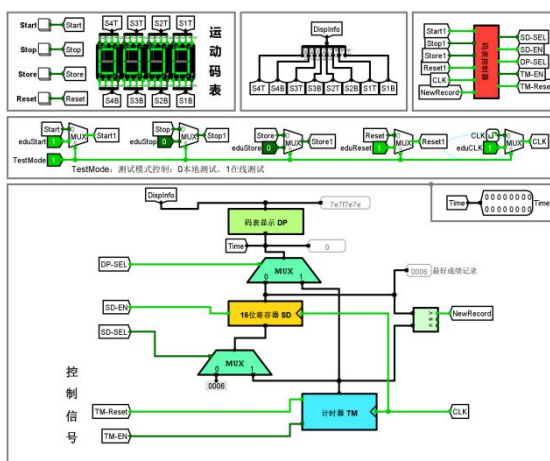


图 1-28 运动码表复位功能

(3) 步骤三：将 circ 文件复制粘贴到相应关卡中，交由平台在线评测。

(4) 步骤四：若通过，检查一下电路是否会发生险象，如果有风险，就增加冗余项消除风险；若未通过，则搜索相关资料后对现有电路进行修改。

1.4 故障与调试

1.4.1 分线器位数顺序问题

分线器会把数据分为若干位，从顶到底依次是数据的某位，但是在需要输出的时候，分线器的位数顺序必须与输入保持一致，否则就会出现部分数据错误，部分正确：



预期输出				实际输出			
Cnt	Din	En	Q	Cnt	Din	En	Q
00	0	0	0	00	0	0	0
01	1	0	0	01	1	0	0
02	2	1	0	02	2	1	0
03	3	1	2	03	3	1	4 Error!
04	4	1	3	04	4	1	c Error!
05	5	0	4	05	5	0	2 Error!
06	6	1	4	06	6	1	2 Error!
07	7	0	6	07	7	0	6
08	8	0	6	08	8	0	6
09	9	0	6	09	9	0	6
0a	a	1	6	0a	a	1	6

图 1-29 4 位并行加载寄存器因分线器顺序错误导致评测失败

1.4.2 部件端口没有连接问题

有些时候看着正确连接了部件，但部件的端口却是蓝色而非正常连接的绿色，问题通常是部件掩盖了线路，导致看上去连上了，但其实并没有。解决方法就是部件删除并重新摆放连接。

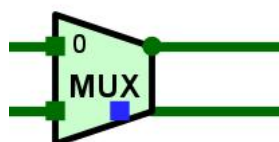


图 1-30 部件端口没有连接

1.5 测试与分析

运动码表的测试在实验步骤中已经说明，本次实验的电路测试工作主要是头歌平台自测运行完成的，此处不过多赘述。

1.6 实验总结

本次实验主要完成了如下几点工作：

(1) 完成了头歌实践教学平台：数字逻辑---运动码表系统设计（Logisim）16 个关卡，成功在 Logisim 中设计出运动码表系统。设计并完成了系统中不同的模块包括：7 段数码管驱动电路、2 选 1 选择器（1 位或 16 位）、无符号比较器设计（4 位或 16 位）、并行加载寄存器（4 位或 16 位）、BCD 计数器状态机及输出函数、BCD 计数器（1 位十进制）、码表计数器（4 位十进制）、码表显示驱动、码表控制器状态机及输出函数、码表控制器、码表数据通路。

(2) 最终运动码表实现的功能：开始计时、停止计时、存储时间、复位、保存或者显示最短时间。

1.7 实验心得

(1) 本次实验我从最开始对 Logisim 使用一窍不通，到后来能熟练拉线路，可以说是收获很多。最开始我不了解电路封装是什么，所以总是找不到任务关卡，所以最开始的几个关卡我都尝试在进入页面时的电路（也就是数码管驱动电路）上实现，虽然能通过第二关，但当输入位数变为 16 位时，显然无法在现有电路上实现。经过网上搜索和线下询问，我才知道要在不同的关卡中实现不同的功能。之后就比较顺利地完成任务，不会的地方就询问 AI 或者上网搜索，往往都能很快完成。总之，本次实验任务锻炼了我应用输入引脚、输出引脚、分线器、基本逻辑门的能力，加深了我对于比较器，选择器，计数器以及级联的认识。

(2) 对实训的建议：有些关卡手动连接工程量太大了，比如 2 路选择器（16 位）和无符号比较器（16 位），且都是重复的连线工作，很容易线路重叠导致看错连错，建议优化此类关卡，减少连线工作量。

2 CPU 设计实验

2.1 设计要求

本次实验任务旨在运用 Logisim 平台内置的运算组件，设计并实现一款 32 位的 MIPS CPU 处理器。实验分为三个部分，分别是设计单周期硬布线控制器 CPU、设计多周期微程序控制器 CPU 以及设计多周期硬布线控制器 CPU。对于构建的 CPU，要求支持 8 条 MIPS 核心指令，最终设计实现的 MIPS 处理器能运行实验包中的冒泡排序测试程序 sort.asm，该程序自动在数据存储器 0~15 号字单元中写入 16 个数据，然后利用冒泡排序将数据升序排序。8 条 MIPS 核心指令如图 2-1 所示：

#	MIPS 指令	RTL 功能描述
1	add \$rd,\$rs,\$rt	$R[\$rd] \leftarrow R[\$rs] + R[\$rt]$ 溢出时产生异常，且不修改 $R[\$rd]$
2	slt \$rd,\$rs,\$rt	$R[\$rd] \leftarrow R[\$rs] < R[\$rt]$ 小于置 1，有符号比较
3	addi \$rt,\$rs,imm	$R[\$rt] \leftarrow R[\$rs] + \text{SignExt}_{16b}(\text{imm})$ 溢出产生异常
4	lw \$rt,imm(\$rs)	$R[\$rt] \leftarrow \text{Mem}_{4B}(R[\$rs] + \text{SignExt}_{16b}(\text{imm}))$
5	sw \$rt,imm(\$rs)	$\text{Mem}_{4B}(R[\$rs] + \text{SignExt}_{16b}(\text{imm})) \leftarrow R[\$rt]$
6	beq \$rs,\$rt,imm	if($R[\$rs] = R[\$rt]$) $PC \leftarrow PC + \text{SignExt}_{18b}(\{imm, 00\})$
7	bne \$rs,\$rt,imm	if($R[\$rs] \neq R[\$rt]$) $PC \leftarrow PC + \text{SignExt}_{18b}(\{imm, 00\})$
8	syscall	系统调用，这里用于停机

图 2-1 8 条 MIPS 核心指令

8 条 MIPS 核心指令的 op(指令的前 6 位)和 func(指令的后 6 位)如图 2-2:

#	MIPS 指令	指令类型	OP (6位)	FUNC (6位)
1	add	R型	000000 (00)	100000 (20)
2	slt	R型	000000 (00)	101010 (2a)
3	addi	I型	001000 (08)	— (无)
4	lw	I型	100011 (23)	— (无)
5	sw	I型	101011 (2b)	— (无)
6	beq	I型	000100 (04)	— (无)
7	bne	I型	000101 (05)	— (无)
8	syscall	特殊R型	000000 (00)	001100 (0c)

图 2-2 指令的 op 和 func

2.2 方案设计

2.2.1 单周期 MIPS CPU（硬布线）及单周期硬布线控制器设计

本关实验内容：利用运算器实验，存储系统实验中构建的运算器、寄存器文件、存储系统等部件以及 Logisim 中其它功能部件，构建一个 32 位 MIPS CPU 单周期处理器。数据通路如图 2-3 所示：

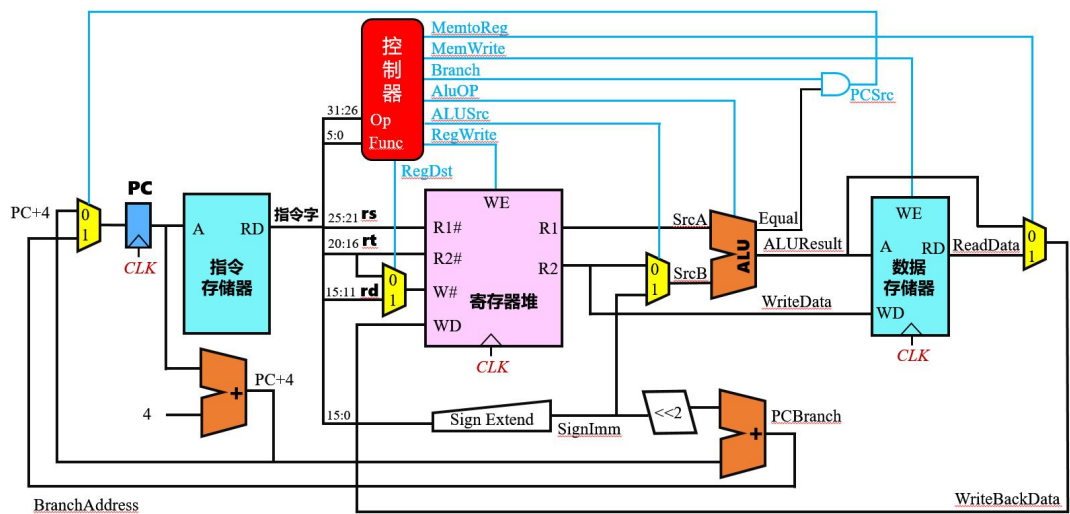


图 2-3 单周期 MIPS CPU 处理器数据通路

电路引脚和功能描述如表 2-1 所示：

表 2-1 单周期 MIPS CPU 处理器电路引脚与功能描述

引脚	输入/输出	位宽	功能描述
CLK	输入	1	时钟信号
PC	输出	32	程序寄存器的值
IR	输出	32	当前指令字
RegWrite	输出	1	寄存器文件写使能控制信号
RDin	输出	32	寄存器文件写入端口的数据
MemWrite	输出	1	存储器写使能控制信号
MDin	输出	32	存储器写入端口的数据

根据数据通路图和输入输出表可以看出有 6 个关键模块：

(1) PC(指令计数器): 其作用是保存下一条待执行指令的内存地址。

- (2) 指令存储器：用于存放 CPU 将要执行的指令序列。
- (3) 数据存储器：负责存取指令在执行期间所需要读写的数据。
- (4) 寄存器堆：包含了 32 个 MIPS 架构的通用寄存器，负责暂存指令需要的操作数和运算结果。
- (5) ALU：运算器，进行各种运算。
- (6) 控制器：对指令进行译码，生成数据通路所需的控制信号。

这些模块中，单周期硬布线控制器的封装电路模块还没有实现，因此需要首先实现。先实现指令译码逻辑和 ALU 控制器逻辑，指令译码逻辑如图 2-4：

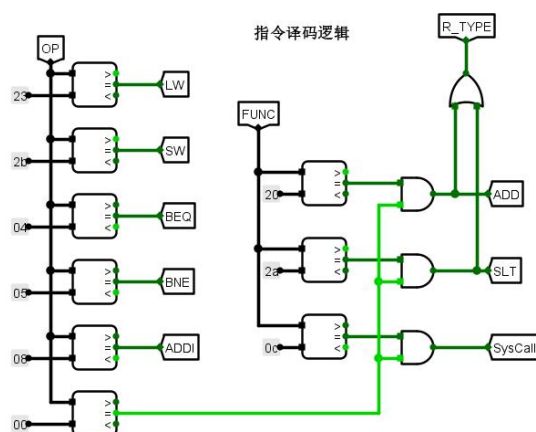


图 2-4 指令译码逻辑图

由图 2-2 得知，当 $op=0x00$ 时，指令为 R 型指令。因此令输入 op 和不同指令的 op 比较，不为 $0x00$ 时可唯一确定指令类型，若为 $0x00$ ，则由 $func$ 唯一确定类型。由于 SysCall 是特殊的 R 型指令，不在 R 型运算指令范畴，R_Type 只用将 ADD+SLT 即可。以上可实现图 2-4 所示逻辑。接下来实现 ALU 控制器逻辑，如图 2-5：

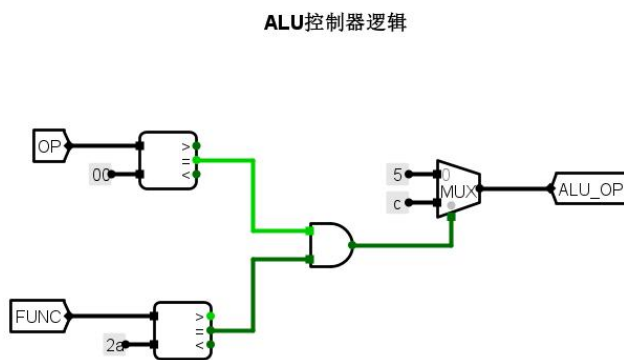


图 2-5 ALU 控制器逻辑

华中科技大学课程实验报告

根据 ALU 内部电路图 2-6，得知 ALU_OP 只有两个选择：5 或 12，当指令为 slt 时，ALU_OP 为 12，其他情况下都是 5，由此可以完成 ALU 控制器逻辑设计。

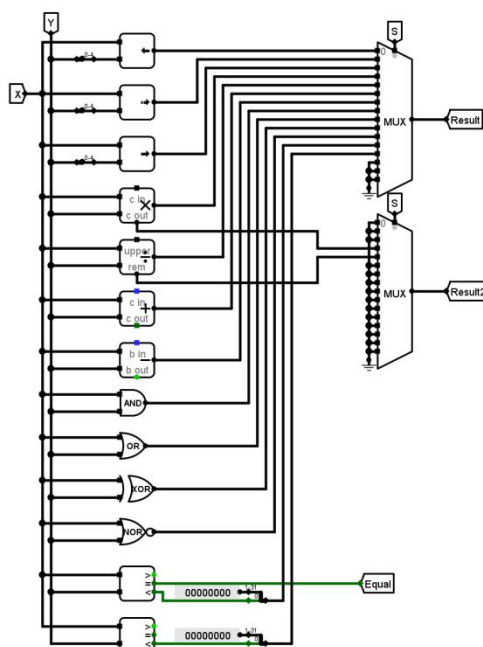


图 2-6 ALU 内部实现

接下来实现指令译码功能，如图 2-7 所示：

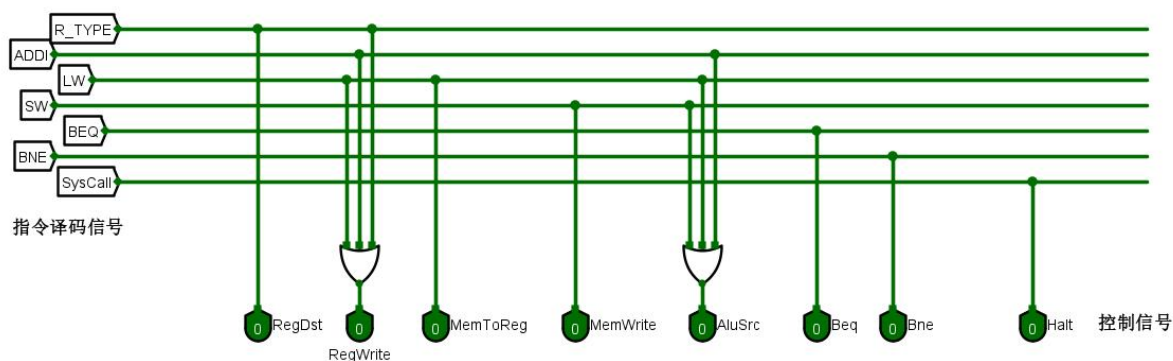


图 2-7 指令译码连线图

根据输出信号反向推导和输入信号的连接：

(1) RegDst: 只有 R 型指令写目标寄存器 rd，所以 RegDst=1 选择 rd；其他指令写 rt，RegDst=0。

(2) RegWrite: R_TYPE + ADDI + LW，只有这三条指令需要把结果写回寄存器堆，RegWrite=1，其他情况下 RegWrite=0。

(3) MemToReg: 只有 lw 指令需要从数据存储器读数据并写回寄存器，所以 MemToReg=1 选择 lw 接入，其他指令 MemToReg=0。

华中科技大学课程实验报告

- (4) MemWrite: 只有 sw 指令需要向数据存储器写入数据。
- (5) AluSrc: LW + SW + ADDI, 这三条指令的 ALU 第二个操作数都来自立即数, 所以 AluSrc=1, 其他情况 AluSrc=0。
- (6) Beq: beq 指令时发出分支信号。
- (7) Bne: bne 指令时发出分支信号。
- (8) Halt: Halt = SysCall, 触发 Halt 信号停止 CPU。由上可以正确连接译码逻辑图。

实现了控制器后即可按照图 2-3 所示的数据通路图正确连接了, 如图 2-8 所示:

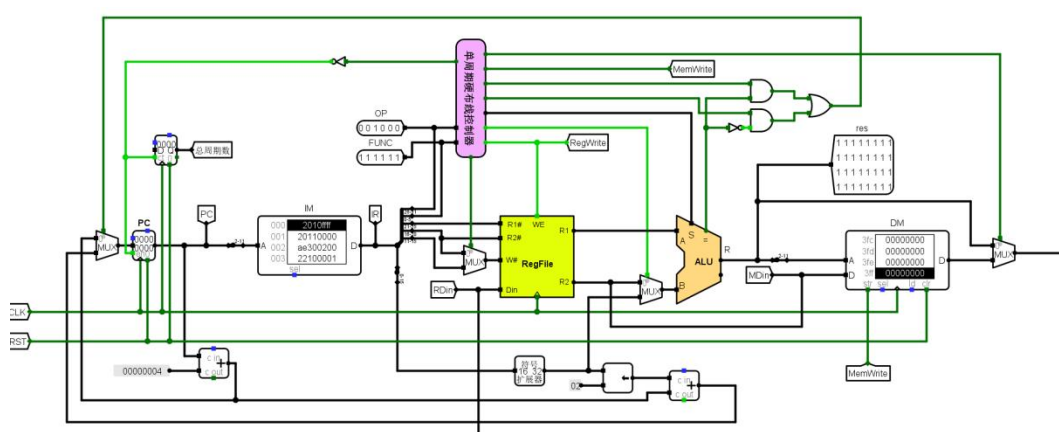


图 2-8 单周期 MIPS CPU (硬布线) 设计图

为了实现图 2-8, 需要注意几个地方的连接:

- (1) 单周期硬布线控制器输出 bne 和 beq, 数据通路图上是 branch 线的集合, 因此需要区别一下 bne 和 beq, 如图 2-9 所示:



图 2-9 bne 和 beq 的实现

- (2) 单周期硬布线控制器输出 halt 信号, 当 halt=1 代表结束 CPU 结束, 虽然数据通路图上没有 halt, 但是可以利用 halt 信号实现对总周期数的计数。如图 2-10。

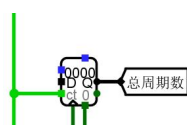


图 2-10 总周期数实现

华中科技大学课程实验报告

2.2.2 多周期 MIPS CPU（微程序）及多周期微程序控制器设计

本关任务依旧是先从设计多周期微程序控制器开始，如图 2-11 所示：

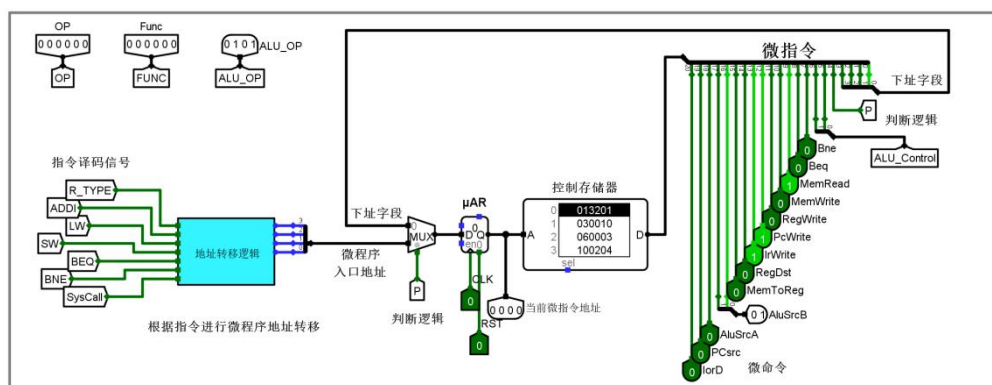


图 2-11 多周期微程序控制器

由图可知在实现控制器前要先实现地址转移逻辑设计，表 2-2 是地址转移真值表。

表 2-2 地址转移真值表

RType	LW	SW	BEQ	BNE	Scall	ADDI	值
1	0	0	0	0	0	0	0111
0	1	0	0	0	0	0	0010
0	0	1	0	0	0	0	0101
0	0	0	1	0	0	0	1001
0	0	0	0	1	0	0	1010
0	0	0	0	0	1	0	1101
0	0	0	0	0	0	1	1011

根据平台测试用例得到的真值表，可以得到输出表达式 $S0 = SYSCALL + BEQ + SW + ADDI + R_Type$ ； $S1 = BNE + LW + ADDI + R_Type$ ； $S2 = SYSCALL + SW + R_Type$ ； $S3 = SYSCALL + BNE + BEQ + ADDI$ 。根据表达式就能生成图 2-22

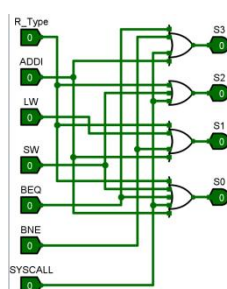


图 2-12 地址转移逻辑图

华中科技大学课程实验报告

接下来该实现微程序控制器中指令译码逻辑和 ALU 控制器逻辑，指令译码逻辑的实现和之前的相同，但是 ALU 控制器逻辑由于多了一种减法控制而略有不同，只需要在多路选择器中添加一种选择，当 $FUNC=20$ 时，运算器做加法；当 $FUNC=2a$ 时，运算器做比较运算。具体实现可见图 2-13。

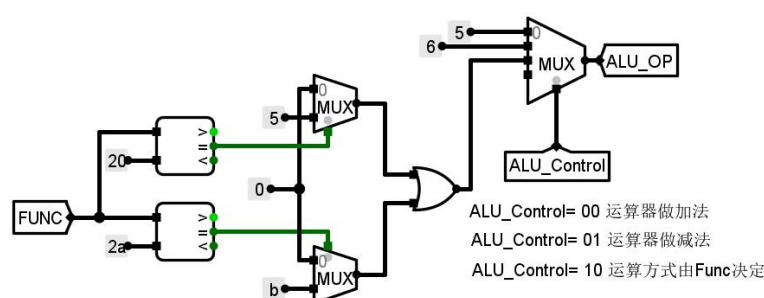


图 2-13 ALU 控制器逻辑图

接下来该实现控制存储器 ROM 的内容的编辑，需要确定状态转化图来确定下址字段，这里直接使用课程平台提供的图，来作为状态转化图，如图 2-14：

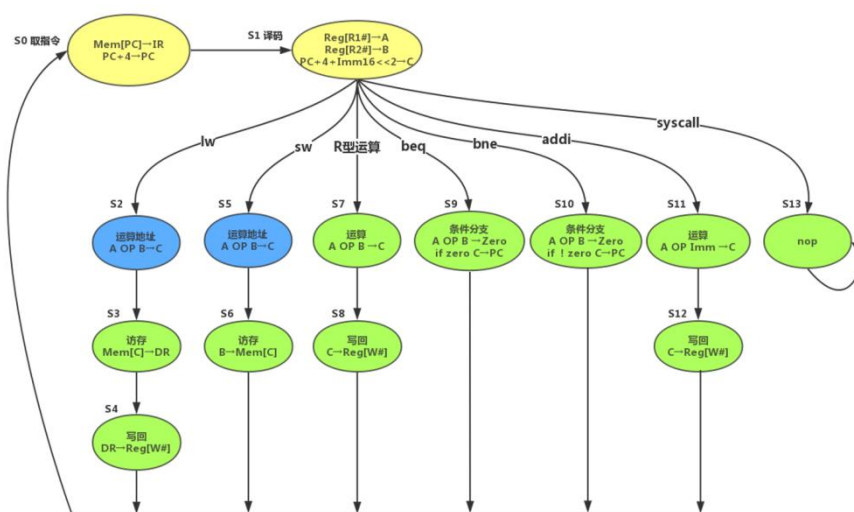


图 2-14 状态转换图

状态设计主要根据输出信号种类，下面列出相较于单周期 CPU 控制器新增的信号：

- ① IorD: 选择存储器地址来源：0 = PC（取指令），1 = ALUOut（访存）；
- ② PCsrc: 选择下一条 PC 的来源：0 = PC+4，1 = 分支目标地址；
- ③ AluSrcA: 选择 ALU 第一个操作数：0 = PC，1 = 寄存器 A；AluSrcB: 选择 ALU 第二个操作数：00 = 寄存器 B，01 = 4，10 = 立即数，11 = 偏移量；
- ④ IRwrite: 指令寄存器写使能：1 = 将存储器读出数据写入 IR；

华中科技大学课程实验报告

⑤ PcWrite:PC 写使能：1 = 允许更新 PC；

⑥ MemRead:存储器读使能：1 = 从存储器读取数据。

有了输出信号种类，可以得出指令控制信号表，如图 2-15 所示：

微指令功能	状态	微指令地址	lOrD	PcSrc	AluSrcA	AluSrcB	MemToReg	RegDst	lRWrite	PcWrite	RegWrite	MemWrite	MemRead	BEQ	BNE	AluControl	P	下址字段
取指令	0	0000	0	0	0	01	0	0	1	1	0	0	1	0	0	00	0	0001
译码	1	0001	0	0	0	11	0	0	0	0	0	0	0	0	0	00	1	0000
LW1	2	0010	0	0	1	10	0	0	0	0	0	0	0	0	0	00	0	0011
LW2	3	0011	1	0	0	00	0	0	0	0	0	0	1	0	0	00	0	0100
LW3	4	0100	0	0	0	00	1	0	0	0	1	0	0	0	0	00	0	0000
SW1	5	0101	0	0	1	10	0	0	0	0	0	0	0	0	0	00	0	0110
SW2	6	0110	1	0	0	00	0	0	0	0	0	1	0	0	0	00	0	0000
R1	7	0111	0	0	1	00	0	0	0	0	0	0	0	0	0	10	0	1000
R2	8	1000	0	0	0	00	0	1	0	0	1	0	0	0	0	00	0	0000
Beq	9	1001	0	1	1	00	0	0	0	0	0	0	0	1	0	01	0	0000
Bne	10	1010	0	1	1	00	0	0	0	0	0	0	0	0	1	01	0	0000
ADDI1	11	1011	0	0	1	10	0	0	0	0	0	0	0	0	0	00	0	1100
ADDI2	12	1100	0	0	0	00	0	0	0	0	1	0	0	0	0	00	0	0000
SYSCALL	13	1101	0	0	0	00	0	0	0	0	0	0	0	0	0	00	0	1101

图 2-15 多周期 MIPS CPU 指令控制信号表

注意表中多的 P 列是分支跳转指令，只有当译码结束，P=1 代表需要跳转指令，执行由地址转移逻辑得到的地址，其他时候顺序执行，无需跳转。

根据信号表可以得出 ROM 内指令的内容，如图 2-16 所示：

微指令	十六进制
0000100110010000000001	13201
0001100000000000010000	30010
0011000000000000000011	60003
100000000001000000100	100204
0000010001000000000000	8800
0011000000000000000110	60006
1000000000100000000000	100400
001000000000001001000	40048
0000001001000000000000	4800
011000000000100100000	C0120
01100000000010100000	C00A0
001100000000000001100	6000C
0000000001000000000000	800
000000000000000001101	D

图 2-16 ROM 内指令内容

接下来需要实现的多周期 MIPS CPU 数据通路如图 2-17 所示：

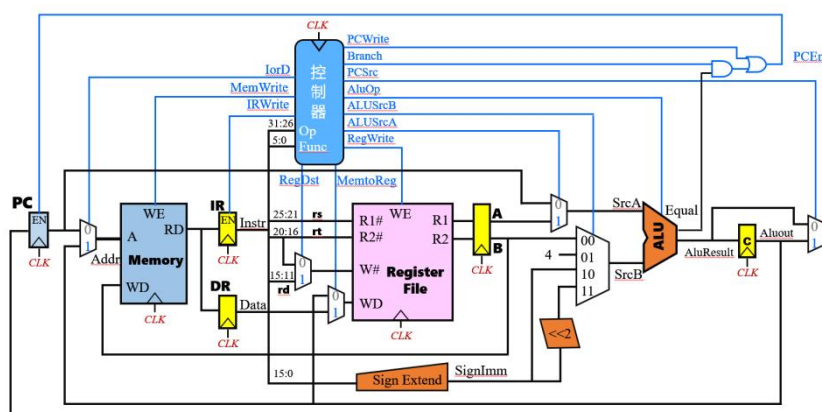


图 2-17 多周期 MIPS CPU 数据通路设计

华中科技大学课程实验报告

需要根据图 2-17 注意到多周期 MIPS CPU 和单周期 MIPS CPU 设计的区别：

- (1) 存储器个数：单周期需要两个存储器，分别是指令寄存器和数据寄存器，多周期只有一个存储器，还有多个暂存寄存器。
- (2) IR/DR 寄存器：单周期极少，多周期必有且带 CLK 输入。
- (3) PC 更新方式：单周期有专门的加法计算部分，多周期通过 ALU 计算。
- (4) 存储器输出个数变化：单周期控制器输出的引脚个数明显少于多周期控制器。

根据这些变化和图 2-17 可以得到多周期 MIPS CPU(微程序)的设计电路图：

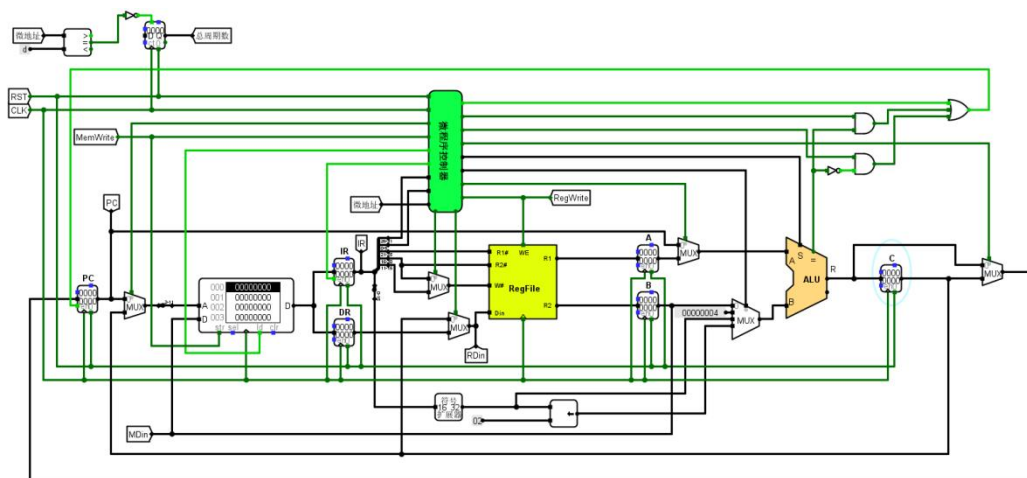


图 2-18 多周期 MIPS CPU 电路图

接下来解释其中关键部分的连接：

- (1) 图 2-17 上控制器输出有条线是 Branch，此处和单周期 CPU 一样，都是 Bne 和 Beq 的集合，此处不再展示具体线路。
- (2) 图 2-17 上控制器输出和图 2-18 上控制器输出个数不匹配，根据之前陈述的多周期 MIPS CPU 相较于单周期 MIPS CPU 增加的控制信号输出，对比之后很容易发现实际电路比数据通路多了 MemRead 输出线路，该线只需要连接到随机存储器 RAM 的 ID 端即可。
- (3) 总周期数的判定可以根据控制器输出的微程序地址，当微程序地址为 d 时，表示此时指令是 SysCall，即 CPU 停止信号，实现如图 2-19 所示。

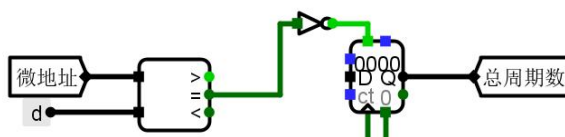


图 2-19 总周期数判定图

华中科技大学课程实验报告

本关任务依旧从设计控制器开始，如图 2-20 所示：

图 2-20 多程序硬布线控制器设计

表 2-3 状态机 FSM 电路引脚与功能描述

状态图和图 2-14 相同, 这里不再展示, 下面推测图 2-14 中 14 个状态数设计四想: 理论上来说最少的状态设计是 S0(取指) → S1(译码) → S2(运算/地址) → S3(访存或写回) → S0, 但这种设计状态内部需要额外判断。所以需要增加的状态来减少分支内部带来的判断: (1) S2(LW 地址)+S5(SW 地址): 控制信号固定, 不用判断是读还是写 (2) S7(R 型)+S11(ADDI): ALUSrcB 信号固定 (3) S9(BEQ)+S10(BNE): 分开跳转条件 (4) S13(Syscall): 单独挂起状态, 系统调用不干扰正常指令流。根据头歌平台上的状态如图 2-14 所示, 14 个状态是最终权衡的结果:

华中科技大学课程实验报告

根据状态转换图得到的状态转换真值表如图 2-21:

S3	S2	S1	S0	现态 10进制	RType	LW	SW	BEQ	BNE	Sca11	ADDI		次态 10进制	N3	N2	N1	N0
0	0	0	0	0	0	0	0	0	0	0	0		1	0	0	0	1
0	0	0	1	1	1	0	0	0	0	0	0		7	0	1	1	1
0	0	0	1	1	0	1	0	0	0	0	0		2	0	0	1	0
0	0	0	1	1	0	0	1	0	0	0	0		5	0	1	0	1
0	0	0	1	1	0	0	0	1	0	0	0		9	1	0	0	1
0	0	0	1	1	0	0	0	0	1	0	0		10	1	0	1	0
0	0	0	1	1	0	0	0	0	0	1	0		13	1	1	0	1
0	0	0	1	1	0	0	0	0	0	0	1		11	1	0	1	1
0	0	1	0	2	0	0	0	0	0	1	0		3	0	0	1	1
0	0	1	1	3	0	0	0	0	1	0	0		4	0	1	0	0
0	1	0	0	4	0	0	0	0	1	1	0		0	0	0	0	0
0	1	0	1	5	0	0	0	1	0	0	0		6	0	1	1	0
0	1	1	0	6	0	0	0	1	0	1	0		0	0	0	0	0
0	1	1	1	7	0	0	0	1	1	0	0		8	1	0	0	0
1	0	0	0	8	0	0	0	1	1	1	0		0	0	0	0	0
1	0	0	1	9	0	0	1	0	0	0	0		0	0	0	0	0
1	0	1	0	10	0	0	1	0	0	1	0		0	0	0	0	0
1	0	1	1	11	0	0	1	0	1	0	0		12	1	1	0	0
1	1	0	0	12	0	0	1	0	1	1	0		0	0	0	0	0
1	1	0	1	13	0	0	1	1	0	0	0		13	1	1	0	1

图 2-21 状态转换表

根据状态表用 Excel 表得到输出函数表达式, 输入后得到的电路图 2-22:

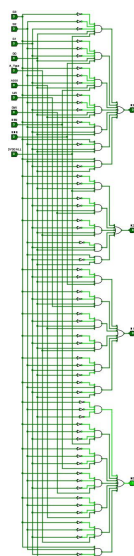


图 2-22 状态机电路图

控制器中 ROM 存储器中存储的指令和多周期微程序 CPU 中一样, 并且两者 CPU 连线除了控制器不同其他都一样, 都是根据数据通路图连接的, 如图 2-23:

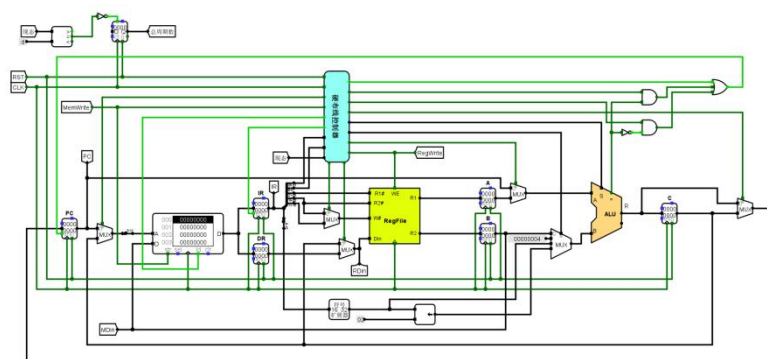


图 2-23 多周期 MIPS CPU(硬布线)设计图

2.3 实验步骤

(1) 加载实验材料, 包括 `cpu.circ`, `cs3410.jar`, `sort.hex`, 单周期硬布线控制器表达式自动生成.xlsx, 微指令自动生成.xlsx, 微程序地址转移逻辑自动生成.xlsx, 硬布线控制器状态转换逻辑自动生成.xlsx。

(2) 根据方案设计和头歌平台任务完成连线和本地测试, 比如冒泡排序测试

(3) 上交头歌平台完成任务。

2.4 故障与调试

2.4.1 数据传输位数问题

PC 是按照字节寻址的, 共 32 位。但是设计过程中使用的 ROM 和 RAM 是按字来存储指令的, 因此需要转化。指令最后两位是 00 确保指令正确, 因此取指令的第 2 位到第 11 位 (从 0 开始编号), 正好是 1024 字节, 所以需要分线器且正确位置连接, 如果错误选择了分线器的位数 (比如第 3 到第 12 位), 就会出现这种错误, 如图 2-24:

预期输出

Cnt	PC	IR	RegW	RDin	MemW	MDin

实际输出

Cnt	PC	IR	RegW	RDin	MemW	MDin
0001	00000004	2010ffff	1	ffffff	0	00000000 Error!

展示原始输出

图 2-24 错误的分线器导致的错误

2.4.2 存储器选择问题

单周期 MIPS (硬布线) 的指令寄存器使用的是 ROM 而非 RAM, 如果使用了 RAM, 就会导致无法加载指令, 出现这种错误, 如图 2-25 所示:

运行结果							
Cnt	PC	IR	RegW	RDin	MemW	MDin	
0000	00000000	00000000	0	00000000	0	00000000	Error!

图 2-25 错误用 RAM 作为指令寄存器

2.5 测试与分析

关卡需要加载冒泡排序程序到指令存储器中，通过运行检查总周期数和参考值是否相同来检验自己电路是否正常。

图 2-26 是单周期 CPU 加载 sort.hex 的操作，图 2-27 是多周期 CPU 加载操作：

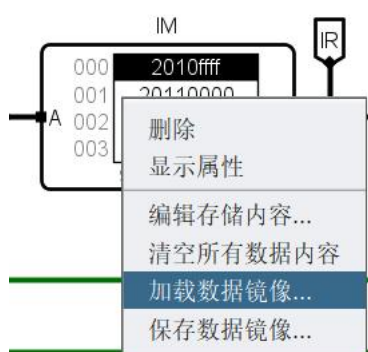


图 2-26 单周期加载操作

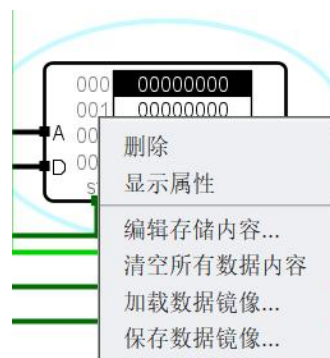


图 2-27 多周期加载操作

图 2-28 是单周期 CPU 运行结果，图 2-29 是多周期 CPU 运行结果：

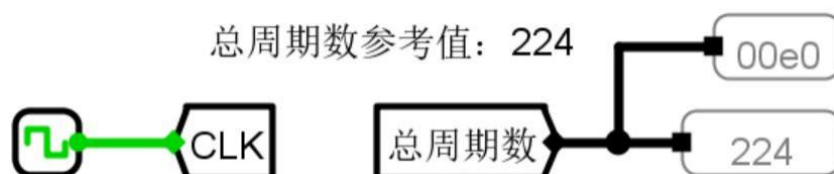


图 2-28 单周期 CPU 运行结果

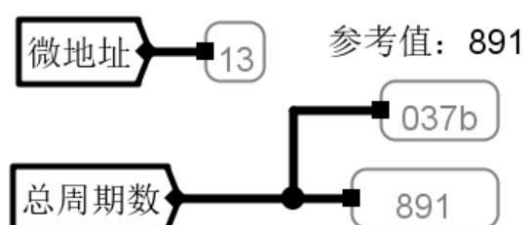


图 2-29 多周期 CPU 运行结果

2.6 实验总结

本次实验主要完成了如下几点工作：

(1) 构建了完整的 CPU 通路，包括单周期 MIPS CPU(硬布线)，多周期 MIPS CPU(微程序)，多周期 MIPS CPU(硬布线)，设计并完成了不同控制器的指令译码，地址转移，状态转换模块。同时也掌握了寄存器连接，ALU 连接，ROM 和 RAM 连接，比较器使用，移位器使用，分线器使用，

(2) 实现的控制器可以根据指令的 op 和 func 以及控制信号得到各个功能组件的信号控制，同时 CPU 可以实现简单的核心 8 条指令译码和执行操作。

2.7 实验心得

(1) 本次实验收获很多，首先是对软件和操作的认识更加深入，了解了 logisim 中可以通过加载数据镜像到存储器的操作，了解了 Excel 表格的各种妙用，了解了 logisim 中多种部件的使用。

(2) 其次巩固了很多理论知识，包括单周期和多周期 CPU 的区别，PC 读取地址和存储器存储地址的区别，硬布线和微程序的区别。学习了很多新知识，比如指令译码过程，指令状态编辑，指令状态转换过程等，对于 RISC 指令集下的 CPU 设计有了初步的了解。

(3) 对课程的建议：建议 CPU 实践课程多出些不同的任务，比如多加一些指令，引入 CISC 等。

• 指导教师评定意见 •

一、原创性声明

本人郑重声明本报告内容，是由作者本人独立完成的。有关观点、方法、数据和文献等的引用已在文中指出。除文中已注明引用的内容外，本报告不包含任何其他个人或集体已经公开发表的作品成果，不存在剽窃、抄袭行为。

特此声明！

作者签字：李金轩

李金轩

二、对课程实验的学术评语（教师填写）

三、对课程实验的评分（教师填写）

评分项目 (分值)	课程目标 1 工具应用 (10 分)	课程目标 2 设计实现 (70 分)	课程目标 3 验收与报告 (20 分)	最终评定 (100 分)
得分				

指导教师签字: _____